Training: Build a Phonebook Application

Objective

Develop a phonebook application where users can add, view, update, and delete contacts. The backend will use **Clojure** with a **MySQL** database, while the frontend will be built using **ClojureScript**. All components will run natively on the host machine without Docker.

Requirements

1. Backend (Clojure + MySQL)

- Use **Ring** as the HTTP server abstraction with **Jetty** as the embedded server.
- Use **Reitit** for data-driven routing and API definition.
- Set up a **MySQL** database to store contacts with the following schema:

```
CREATE TABLE contacts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  phone_number VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE,
  address TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- Implement RESTful APIs with the following endpoints:
  - `GET /api/contacts` : Retrieve all contacts.
  - `POST /api/contacts` : Add a new contact.
  - `GET /api/contacts/:id` : Retrieve a specific contact by ID.
  - `PUT /api/contacts/:id` : Update a specific contact by ID.

    - `DELETE /api/contacts/:id` : Delete a specific contact by ID.
- Use **next.jdbc** for database interactions and connection pooling with **HikariCP**.
- Use **Clojure spec** or **Malli** to validate input data (e.g., ensure phone numbers are in a valid format and name is present).
- Enable CORS so the ClojureScript frontend can communicate with the backend.

2. Frontend (ClojureScript)

- Build a simple, responsive UI using **ClojureScript** with **Reagent** (a minimal React wrapper).
- Use **Shadow-cljs** as the build tool for fast compilation and hot code reloading.
- Include the following pages/components:
    - **Contact List Page** : Displays all contacts in a table or card layout. Each contact should show their name, phone number, email (if available), and address (if available).
    - **Contact Detail Page** : Shows detailed information about a specific contact and allows updating the contact's details.
    - **Create Contact Form** : Allows users to add new contacts.
- Use `cljs-ajax` or the Fetch API to communicate with the backend APIs.
- Add basic form validation (e.g., name and phone number are required, phone number must follow a valid format).

3. Local Setup (No Docker)

- Install **MySQL Server** locally and create a database named `phonebook_db`.
- Configure the Clojure backend to connect to the local MySQL instance via a configuration file or environment variables (e.g., `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`).
- Use **deps.edn** or **project.clj** to manage Clojure/ClojureScript dependencies.
- Start the backend server locally (e.g., `clj -M:run` or `lein run`).

- Start the ClojureScript development server using Shadow-cljs (e.g., `npx shadow-cljs watch app`).
- Ensure the frontend can reach the backend API (e.g., `http://localhost:3000`).

4. Optional Enhancements

- Add search functionality to allow users to search contacts by name or phone number.
- Add pagination to the contact list page for better performance with large datasets.
- Use **Re-frame** for state management in the frontend.
- Write unit tests for the backend using **Clojure.test** and integration tests with a test database.
- Write frontend tests using **cljs.test** and Shadow-cljs test runners.
- Add a CI/CD pipeline configuration (e.g., GitHub Actions) to automate testing and deployment.

Deliverables

- A working phonebook application with all required features.
- Source code for the backend, frontend, and database schema/migrations.
- A `README.md` file with instructions on how to:
    - Set up the development environment (install Java, Clojure CLI, Node.js, and MySQL).
    - Run the application locally.
    - Test the application.

Hints

- Start by setting up your local MySQL database and verifying the connection from a simple Clojure REPL session using `next.jdbc`.
- Define your API routes with Reitit first, then implement the handler functions one by one.

- Use Reagent's atom (`reagent.core/atom`) for component-local state, or introduce Re-frame if you need more structured global state management.
- Keep validation logic pure and reusable so it can be shared between the frontend and backend if possible.
- Test each API endpoint with `curl` or a REST client before wiring it up to the UI.